Lecture 7 — Number Theory for Regionals

> Regional Level · Prerequisites: Year Curriculum Weeks 10–11 (gcd, sieve, modular arithmetic, inverses) and 30 (inclusion–exclusion) · Estimated time: 2.5 hours

1. Overview

Number theory problems at regionals are usually medium difficulty: once the right identity is seen, the code is short. The difficulty is (a) recognising the identity in a story about gears, calendars, or lattice points and (b) not overflowing 64-bit integers. This lecture covers the toolkit that solves most of them:

- the extended Euclidean algorithm and linear Diophantine equations,
- the Chinese Remainder Theorem for **non-coprime** moduli,
- Euler's totient φ and its use in gcd-counting,
- inclusion–exclusion over prime factors,
- the Möbius function for counting coprime pairs, and the "exact gcd" sieve.

Advanced topics (Pollard's rho, Dirichlet convolutions, min-25) belong to later tracks.

2. Extended Euclid and Diophantine equations

2.1 Extended Euclid

`extgcd(a, b)` returns `g = gcd(a, b)` and integers `x, y` with `a·x + b·y = g`. It follows from `gcd(a, b) = gcd(b, a mod b)`: if `b·x₁ + (a mod b)·y₁ = g`, then with `a mod b = a − ⌊a/b⌋·b` we get `x = y₁`, `y = x₁ − ⌊a/b⌋·y₁`. The returned coefficients satisfy `|x| ≤ b/g` and `|y| ≤ a/g` (for `a, b > 0`), so the recursion itself never overflows.

The most common use: the inverse of `a` modulo `m` exists iff `gcd(a, m) = 1`, and it is `x mod m` from `extgcd(a, m)`. Unlike Fermat's little theorem, this works for composite `m`.

2.2 Linear Diophantine equations

`a·x + b·y = c` has integer solutions iff `g = gcd(a, b)` divides `c`. From `a·x' + b·y' = g`, a particular solution is `x₀ = x'·(c/g)`, `y₀ = y'·(c/g)`, and all solutions are

```
x = x₀ + k·(b/g),   y = y₀ − k·(a/g),   k ∈ ℤ.
```

To count solutions with `x ∈ [x_lo, x_hi]` and `y ∈ [y_lo, y_hi]`, convert each constraint into an interval of `k` (careful with floor/ceil of negative numbers and with the sign of `a/g`, `b/g`), then intersect the intervals. The product `x'·(c/g)` can be about $10^{18} \cdot 10^9$, so use `__int128` or reduce `x'` modulo `b/g` first. *Codeforces 7C Line* (find any integer point on `Ax + By + C = 0`) is the direct application, and has the usual sign traps.

3. The Chinese Remainder Theorem

3.1 From stories to congruences

"Light A blinks every 12 seconds and blinked at second 5; light B every 18 seconds and blinked at second 11; when do they blink together?" is the system `x ≡ 5 (mod 12)`, `x ≡ 11 (mod 18)`. Moduli in such stories are rarely coprime, so the textbook CRT formula (which assumes pairwise coprime moduli) is not enough.

3.2 General CRT

A system `x ≡ r₁ (mod m₁)`, `x ≡ r₂ (mod m₂)` is solvable iff `r₁ ≡ r₂ (mod g)` with `g = gcd(m₁, m₂)`, and then the solution is unique modulo `lcm(m₁, m₂)`. Derivation: write `x = r₁ + m₁·t`. We need `m₁·t ≡ r₂ − r₁ (mod m₂)`. If `g ∤ (r₂ − r₁)` there is no solution. Otherwise divide by `g`: `(m₁/g)·t ≡ (r₂ − r₁)/g (mod m₂/g)`, where `m₁/g` and `m₂/g` are coprime. If `extgcd(m₁, m₂)` returns `p` with `m₁·p + m₂·q = g`, then `(m₁/g)·p ≡ 1 (mod m₂/g)`, so `t ≡ ((r₂ − r₁)/g)·p (mod m₂/g)`. Finally `x = r₁ + m₁·t` modulo the lcm.

With many congruences, start from `x ≡ 0 (mod 1)` and merge them one at a time. The lcm can grow quickly; problems usually guarantee it fits in 64 bits, but intermediate products need `__int128`.

The example above: `gcd(12, 18) = 6` and `11 − 5 = 6` is divisible by 6, so a solution exists modulo 36. The merge gives `x ≡ 29 (mod 36)` (check: 29 = 2·12 + 5 = 18 + 11).

Related: *Codeforces 687B Remainders Game* asks whether knowing `x mod c_i` for all `i` determines `x mod k`; by CRT this holds iff `k` divides `lcm(c_i)`, which you check without computing the (huge) lcm by maintaining `L = lcm(L, gcd(c_i, k))`, a divisor of `k` that never overflows; the answer is yes iff finally `L = k`.

4. Euler's totient

`φ(n)` counts `1 ≤ k ≤ n` coprime to `n`, and `φ(n) = n·Π_{p | n}(1 − 1/p)`. Compute a single value by trial division in O(√n), or all values up to `N` with a sieve.

4.1 Counting by gcd

For a divisor d of n, the numbers $1 \le i \le n$ with $\gcd(i, n) = d$ are exactly $i = d \cdot j$ with $1 \le j \le n/d$ and $\gcd(j, n/d) = 1$; there are $\varphi(n/d)$ of them. Two consequences:

- $\sum_{d \mid n} \varphi(d) = n$ (every i has some gcd with n).
- $\sum_{i=1}^{n} \gcd(i, n) = \sum_{d \mid n} d \cdot \varphi(n/d)$.

For $n \le 10^{12}$, list divisors in $O(\sqrt{n})$ (at most 6720 of them) and compute each φ. *Codeforces 1295D Same GCDs* asks for the number of $0 \le x < m$ with $\gcd(a, m) = \gcd(a + x, m)$. As x ranges over $[0, m)$, $a + x$ ranges over a complete residue system mod m, so the answer is the number of y in $[0, m)$ with $\gcd(y, m) = g = \gcd(a, m)$, which is $\varphi(m/g)$.

5. Inclusion–exclusion over prime factors

$|A_1 \cup \dots \cup A_k| = \sum_{\varnothing \ne S} (-1)^{|S|+1} \left|\bigcap_{i \in S} A_i\right|$. With A_i = multiples of prime p_i up to n, the intersection over S is the multiples of $\prod_{i \in S} p_i$, of which there are $\lfloor n / \prod \rfloor$.

CSES 2185 Prime Multiples: $n \le 10^{18}$, $k \le 20$ primes. Iterate over all 2^k masks. The danger is overflow of the product: check $\text{prod} > n / p$ **before** multiplying; if the product exceeds n, the term is zero and can be skipped.

The same pattern counts integers in $[1, M]$ coprime to n: factor n (at most 15 distinct primes for $n \le 10^{18}$, at most 11 for $n \le 10^{12}$), then $\sum_{S} (-1)^{|S|} \lfloor M / \prod_S p \rfloor$. Wrapping this in a binary search on M finds the k-th number coprime to n.

6. The Möbius function

$\mu(1) = 1$; $\mu(n) = (-1)^k$ if n is a product of k distinct primes; $\mu(n) = 0$ if n has a squared prime factor. The key identity is

```
Σ_{d | n} μ(d) = [n = 1].
```

It is inclusion–exclusion in disguise: the terms with $\mu(d) \ne 0$ are the subsets of the distinct primes of n.

6.1 Counting coprime pairs (CSES 2417)

Given $n \le 10^5$ numbers up to 10^6, count pairs with gcd 1. Write $[\gcd(a, b) = 1] = \sum_{d \mid \gcd(a,b)} \mu(d)$ and swap the sums:

```
#coprime pairs = Σ_d μ(d) · C(c_d, 2),   c_d = #{i : d | x_i}.
```

Compute `c_d` for every `d` by summing a frequency array over multiples of `d`, which is a harmonic sum O(V log V) for `V = 10⁶`. Compute μ with a linear sieve.

6.2 The "exact gcd" sieve

An alternative that avoids μ: let `F(d)` be the number of pairs whose gcd is a multiple of `d` (that is, `C(c_d, 2)`), and `G(d)` the number with gcd exactly `d`. Then `G(d) = F(d) − Σ_{k ≥ 2} G(k·d)`; compute it for `d` from `V` down to 1. The answer is `G(1)`, but you also get every `G(d)`, which answers "sum of gcd over all pairs" and similar questions. *Codeforces 803F Coprime Subsequences* is the same idea with `F(d) = 2^{c_d} − 1` non-empty subsets.

7. Reference implementations

7.1 General CRT

Input: `k`, then `k` lines `r m`. Output: the smallest non-negative solution and the combined modulus, or `no solution`.

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128;

ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

bool crtMerge(ll& r1, ll& m1, ll r2, ll m2) {
    ll p, q;
    ll g = extgcd(m1, m2, p, q);                  // m1*p + m2*q = g
    if ((r2 - r1) % g != 0) return false;
    ll lcm = m1 / g * m2;                         // assumed to fit in long long
    ll mod2 = m2 / g;
    i128 t = (i128)((r2 - r1) / g) * p % mod2;
    if (t < 0) t += mod2;
    i128 x = ((i128)r1 + (i128)m1 * t) % lcm;
    if (x < 0) x += lcm;
    r1 = (ll)x;
    m1 = lcm;
    return true;
}

int main() {
```

```
    int k;
    cin >> k;
    ll r = 0, m = 1;
    bool ok = true;
    for (int i = 0; i < k; i++) {
        ll ri, mi;
        cin >> ri >> mi;
        ri = ((ri % mi) + mi) % mi;
        if (ok) ok = crtMerge(r, m, ri, mi);
    }
    if (!ok) cout << "no solution\n";
    else cout << r << ' ' << m << '\n';
}
```

Verified against brute force on random systems with moduli up to 30. Input `2 3`, `3 5`, `2 7` gives `23 105`; `1 4`, `3 6` gives `9 12`; `1 4`, `2 6` gives `no solution`.

7.2 Σ gcd(i, n) for n ≤ 10¹²

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll phi(ll n) {
    ll result = n;
    for (ll p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            result -= result / p;
        }
    }
    if (n > 1) result -= result / n;
    return result;
}

int main() {
    ll n;
    cin >> n;
    ll total = 0;                       // <= n * d(n) <= 1e12 * 6720 < 2^63
    for (ll d = 1; d * d <= n; d++) {
        if (n % d) continue;
        total += d * phi(n / d);
        if (d != n / d) total += (n / d) * phi(d);
    }
    cout << total << '\n';
}
```

For `n = 12` it prints 40 (gcds 1,2,3,4,1,6,1,4,3,2,1,12). It runs in a few milliseconds even for `n = 963761198400`, the number below 10¹² with the most divisors.

7.3 CSES 2185 — Prime Multiples

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ll n;
    int k;
    cin >> n >> k;
    vector<ll> p(k);
    for (auto& x : p) cin >> x;
    ll total = 0;
    for (int mask = 1; mask < (1 << k); mask++) {
        ll prod = 1;
        bool tooBig = false;
        for (int i = 0; i < k && !tooBig; i++) {
            if (!(mask >> i & 1)) continue;
            if (prod > n / p[i]) tooBig = true;     // prod * p[i] > n
            else prod *= p[i];
        }
        if (tooBig) continue;
        ll cnt = n / prod;
        total += (__builtin_popcount(mask) & 1) ? cnt : -cnt;
    }
    cout << total << '\n';
}
```

7.4 CSES 2417 — Counting Coprime Pairs

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    const int MAXV = 1'000'000;
    vector<int> freq(MAXV + 1, 0);
    for (int i = 0; i < n; i++) { int x; cin >> x; freq[x]++; }
    vector<int> mu(MAXV + 1, 0), primes;
    vector<bool> composite(MAXV + 1, false);
    mu[1] = 1;
    for (int i = 2; i <= MAXV; i++) {                // linear sieve
        if (!composite[i]) { primes.push_back(i); mu[i] = -1; }
        for (int p : primes) {
            if ((ll)i * p > MAXV) break;
            composite[i * p] = true;
            if (i % p == 0) { mu[i * p] = 0; break; }
            mu[i * p] = -mu[i];
```

```
        }
    }
    ll answer = 0;
    for (int d = 1; d <= MAXV; d++) {
        if (mu[d] == 0) continue;
        ll c = 0;
        for (int m = d; m <= MAXV; m += d) c += freq[m];
        answer += mu[d] * c * (c - 1) / 2;
    }
    cout << answer << '\n';
}
```

Verified against an $O(n^2)$ gcd check on random inputs; the CSES sample gives 19.

8. Common mistakes

- **Overflow in modular multiplication** with 64-bit moduli: use `__int128`.
- **Negative remainders**: C++ `%` keeps the sign of the dividend; normalise with `((x % m) + m) % m`.
- **Assuming coprime moduli** in CRT.
- **Forgetting the last prime** after trial division (`if (n > 1)`).
- **Overflowing products** in inclusion–exclusion; divide before multiplying.
- **Floor division of negatives** when converting Diophantine constraints to ranges of `k`: `a / b` in C++ truncates toward zero.
- **Linear-sieve slip**: when `p | i`, set `mu[i·p] = 0` and break.
- **Sieving to the wrong bound**: values up to 10^6 but sieve to 10^5.

9. Practice problems

- Codeforces 7C — Line: https://codeforces.com/problemset/problem/7/C
- Kattis `chineseremainder`: https://open.kattis.com/problems/chineseremainder
- Kattis `generalchineseremainder`: https://open.kattis.com/problems/generalchineseremainder
- Kattis `relatives` (Euler φ): https://open.kattis.com/problems/relatives
- Codeforces 687B — Remainders Game: https://codeforces.com/problemset/problem/687/B
- Codeforces 1295D — Same GCDs: https://codeforces.com/problemset/problem/1295/D
- CSES 2185 — Prime Multiples: https://cses.fi/problemset/task/2185
- CSES 2417 — Counting Coprime Pairs: https://cses.fi/problemset/task/2417
- Codeforces 803F — Coprime Subsequences: https://codeforces.com/problemset/problem/803/F
- CSES 1713 — Counting Divisors: https://cses.fi/problemset/task/1713
- CSES 1082 — Sum of Divisors: https://cses.fi/problemset/task/1082

10. Further reading

- cp-algorithms: "Extended Euclidean Algorithm", "Linear Diophantine Equations", "Chinese Remainder Theorem", "Euler's totient function", "Möbius function" (https://cp-algorithms.com).
- Ivan Niven, Herbert Zuckerman, Hugh Montgomery — *An Introduction to the Theory of Numbers*, Chapters 2 and 4.
- Graham, Knuth, Patashnik — *Concrete Mathematics*, Chapter 4 (number theory, including Möbius inversion).
- World Finals track Lecture 7 continues with Möbius inversion and Dirichlet techniques.